

Optimization

6.4 The ε -constraint method

José Firmino Aguilar Madeira

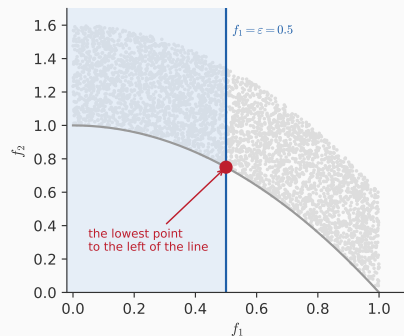
Another idea: optimize one, bound the other

In 6.3, with $f_1 = x$, $f_2 = 1 - x^2$, the weighted sum only found the two extremes. What if, instead of combining the objectives, we bound one of them?

$$\min_{x \in X} f_2(x) \quad \text{s.t.} \quad f_1(x) \leq \varepsilon$$

Among the solutions with acceptable f_1 , the one with the smallest f_2 .

In the objective space: the vertical line $f_1 = \varepsilon$ and the lowest point to its left.



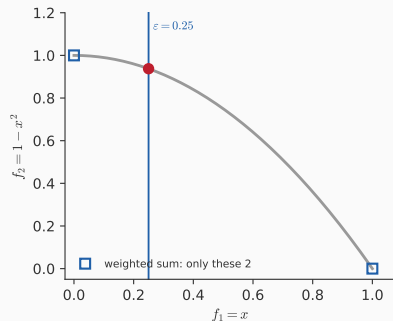
The nonconvex example from 6.2 (Y in grey).

The same example as in 6.3, by hand

$$\min f_2 = 1 - x^2 \quad \text{s.t.} \quad f_1 = x \leq \varepsilon, \quad 0 \leq x \leq 1.$$

Since $1 - x^2$ decreases as x increases: $x^* = \varepsilon$.

$$\varepsilon = 0.25 \Rightarrow (f_1, f_2) = (0.25, 0.9375)$$



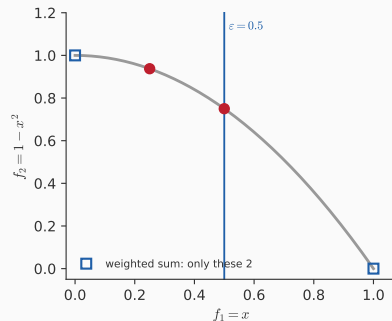
The same example as in 6.3, by hand

$$\min f_2 = 1 - x^2 \quad \text{s.t.} \quad f_1 = x \leq \varepsilon, \quad 0 \leq x \leq 1.$$

Since $1 - x^2$ decreases as x increases: $x^* = \varepsilon$.

$$\varepsilon = 0.25 \Rightarrow (f_1, f_2) = (0.25, 0.9375)$$

$$\varepsilon = 0.50 \Rightarrow (f_1, f_2) = (0.50, 0.75)$$



The same example as in 6.3, by hand

$$\min f_2 = 1 - x^2 \quad \text{s.t.} \quad f_1 = x \leq \varepsilon, \quad 0 \leq x \leq 1.$$

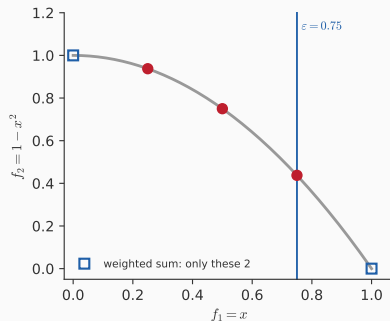
Since $1 - x^2$ decreases as x increases: $x^* = \varepsilon$.

$$\varepsilon = 0.25 \Rightarrow (f_1, f_2) = (0.25, 0.9375)$$

$$\varepsilon = 0.50 \Rightarrow (f_1, f_2) = (0.50, 0.75)$$

$$\varepsilon = 0.75 \Rightarrow (f_1, f_2) = (0.75, 0.4375)$$

These three points were impossible to obtain with the weighted sum.



The same example as in 6.3, by hand

$$\min f_2 = 1 - x^2 \quad \text{s.t. } f_1 = x \leq \varepsilon, \quad 0 \leq x \leq 1.$$

Since $1 - x^2$ decreases as x increases: $x^* = \varepsilon$.

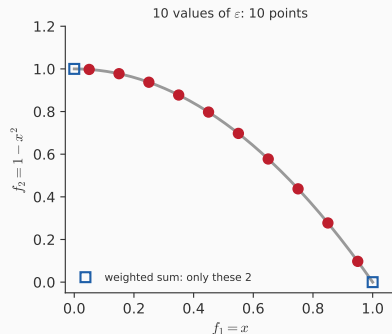
$$\varepsilon = 0.25 \Rightarrow (f_1, f_2) = (0.25, 0.9375)$$

$$\varepsilon = 0.50 \Rightarrow (f_1, f_2) = (0.50, 0.75)$$

$$\varepsilon = 0.75 \Rightarrow (f_1, f_2) = (0.75, 0.4375)$$

What if we keep moving the line?

$\varepsilon = 0.05, 0.15, \dots, 0.95$: the front is practically swept.



The same example as in 6.3, by hand

$$\min f_2 = 1 - x^2 \text{ s.t. } f_1 = x \leq \varepsilon, \quad 0 \leq x \leq 1.$$

Since $1 - x^2$ decreases as x increases: $x^* = \varepsilon$.

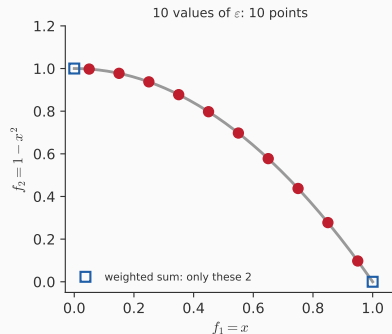
$$\varepsilon = 0.25 \Rightarrow (f_1, f_2) = (0.25, 0.9375)$$

$$\varepsilon = 0.50 \Rightarrow (f_1, f_2) = (0.50, 0.75)$$

$$\varepsilon = 0.75 \Rightarrow (f_1, f_2) = (0.75, 0.4375)$$

What if we keep moving the line?

$\varepsilon = 0.05, 0.15, \dots, 0.95$: the front is practically swept.



weighted sum: 10 000 weights
→ 2 points

vs.

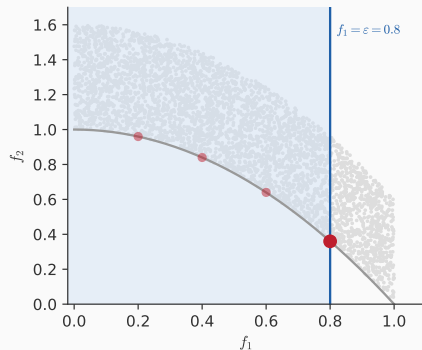
ε -constraint: 10 values of $\varepsilon \rightarrow$
10 points

Why does it work?

The weighted sum used a **tilted ruler**, which cannot touch the concave interior first.

Here the line $f_1 = \varepsilon$ is **vertical**: it can cut the region at any position, and we choose the lowest point to its left.

ε directly controls how much we accept in f_1 : if the constraint is active, $f_1(x^*) = \varepsilon$. In 6.3, $w \rightarrow$ a position hard to predict.



$\varepsilon = 0.2, 0.4, 0.6, 0.8$ on the nonconvex example
from 6.2.

The general formulation

Haimes, Lasdon and Wismer (1971). With m objectives, we choose one, which we number f_1 (in the example, it was f_2):

$$\min_{x \in X} f_1(x) \quad \text{s.t.} \quad f_i(x) \leq \varepsilon_i, \quad i \neq 1$$

- One objective remains an objective; the other $m - 1$ become **constraints**: it is a problem from Ch. 4.
- The ε_i have **the units of f_i** : they are bounds on the objectives, not numerical tolerances.
- Sweep: $\varepsilon_i \in [z_i^{id}, \hat{z}_i^{nad}]$ (6.2). Below z_i^{id} : infeasible.

**Nantes–Lille: the cheapest,
with duration $\leq \varepsilon$**

$\varepsilon = 4$ h plane (134 €)

$\varepsilon = 5$ h TGV low-cost (40 €)

$\varepsilon = 10$ h bus 2 (28 €)

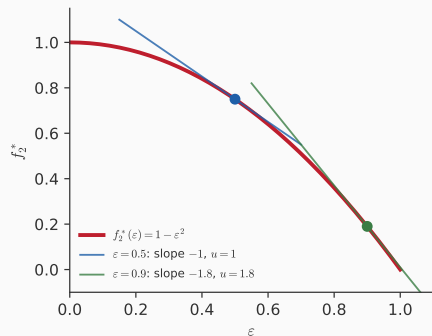
ε has a **physical meaning**: it is often easier for the decision-maker than a weight.

The multiplier: the trade-off rate

In the example, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. In the course's form (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, and the multiplier of $f_1 \leq \varepsilon$ is

$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

At $\varepsilon = 0.5$: $x^* = 0.5$, $f_2^* = 0.75$, $u = 1$.



The multiplier: the trade-off rate

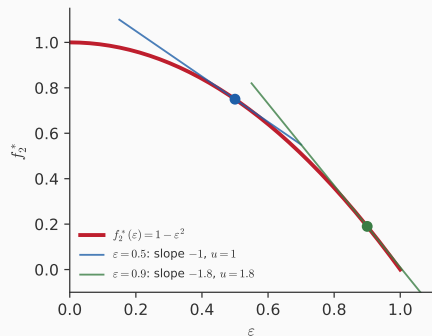
In the example, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. In the course's form (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, and the multiplier of $f_1 \leq \varepsilon$ is

$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

At $\varepsilon = 0.5$: $x^* = 0.5$, $f_2^* = 0.75$, $u = 1$.

If we accept increasing f_1 from 0.50 to 0.51, how much do we gain in f_2 ?

With $u = 1$: $\Delta f_2^* \approx -0.01$.



The multiplier: the trade-off rate

In the example, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. In the course's form (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, and the multiplier of $f_1 \leq \varepsilon$ is

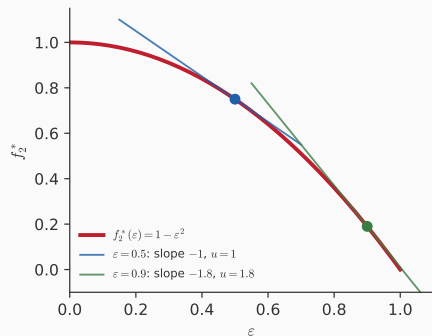
$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

At $\varepsilon = 0.5$: $x^* = 0.5$, $f_2^* = 0.75$, $u = 1$.

If we accept increasing f_1 from 0.50 to 0.51, how much do we gain in f_2 ?

With $u = 1$: $\Delta f_2^* \approx -0.01$.

Exact check: $f_2(0.51) = 1 - 0.51^2 = 0.7399$;
difference -0.0101 . At $\varepsilon = 0.9$, $u = 1.8$: the same
slack buys ≈ 0.018 .



The multiplier: the trade-off rate

In the example, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. In the course's form (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, and the multiplier of $f_1 \leq \varepsilon$ is

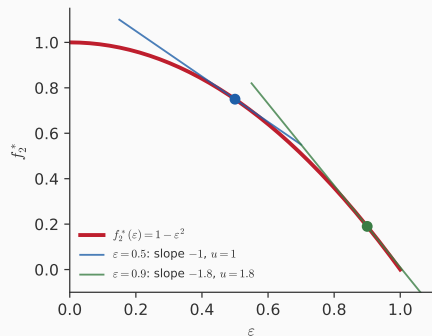
$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

At $\varepsilon = 0.5$: $x^* = 0.5$, $f_2^* = 0.75$, $u = 1$.

If we accept increasing f_1 from 0.50 to 0.51, how much do we gain in f_2 ?

With $u = 1$: $\Delta f_2^* \approx -0.01$.

Exact check: $f_2(0.51) = 1 - 0.51^2 = 0.7399$; difference -0.0101 . At $\varepsilon = 0.9$, $u = 1.8$: the same slack buys ≈ 0.018 .



u is the **local trade-off rate** between objectives — valuable information for the decision-maker.

What can we guarantee?

With the subproblem solved globally:

- if the solution x^* is **unique**, it is **efficient**;
- any efficient x^* is recovered with $\varepsilon_i = f_i(x^*)$ ($i \neq 1$)
(Chankong and Haimes, 1983) — convex or not.

In principle it recovers **the whole front**,
non-supported included — **if the subproblems are**
solved globally.

Without uniqueness, ties are possible: another optimal solution with the same f_1 and better in a bounded objective. Only *weak* efficiency is guaranteed (an x strictly better in all would be feasible with smaller f_1). For efficiency: augmented ε -constraint or tie-breaking.

What can we guarantee?

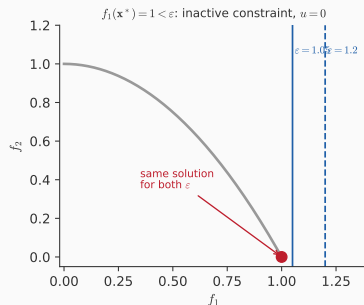
With the subproblem solved **globally**:

- if the solution x^* is **unique**, it is **efficient**;
- any efficient x^* is recovered with $\varepsilon_i = f_i(x^*)$ ($i \neq 1$) (Chankong and Haimes, 1983) — convex or not.

In principle it recovers **the whole front**,
non-supported included — **if the subproblems are
solved globally**.

Without uniqueness, ties are possible: another optimal solution with the same f_1 and better in a bounded objective. Only *weak* efficiency is guaranteed (an x strictly better in all would be feasible with smaller f_1). For efficiency: augmented ε -constraint or tie-breaking.

Inactive constraint. If $f_i(x^*) < \varepsilon_i$, then $u_i = 0$: **the bound is not determining the solution** — moving the line does not change it.



The price of the ε -constraint

Advantages

- finds *supported* and *non-supported* points (with subproblems solved globally);
- ε_i with a direct physical interpretation;
- better control of where to search;
- multipliers = trade-off rates;
- reuses the constrained methods of Ch. 4.

Limitations

- one optimization **per point**;
- each subproblem gains $m - 1$ constraints;
- ε too demanding $\rightarrow$ infeasible;
- constraints may become inactive;
- in nonconvex problems, solving the subproblems locally does not preserve the global guarantees;
- cost grows quickly with m .

k values for each of the $m - 1$ bounds: $\boxed{k^{m-1} \text{ problems}}$ $k = 10: \quad m = 2 \rightarrow 10; \quad m = 3 \rightarrow 100;$
 $m = 5 \rightarrow 10\,000.$

In MATLAB: sweeping ε with fmincon

The core:

```
f1 = @(x) x(1); f2 = @(x) 1 - x(1)^2 + x(2);  
lb = [0 0]; ub = [1 0.6]; x0 = [0.5 0.3];  
info.nfev = 0;  
for e = 0.05:0.1:0.95  
    nonl = @(x) deal(f1(x) - e, []); % f1 <= e  
    [x, fval, ~, out, lambda] = fmincon(f2, x0, ...  
                                       [], [], [], [], lb, ub, nonl);  
    u = lambda.ineqnonlin; % trade-off rate  
    info.nfev = info.nfev + out.funcCount;  
    x0 = x; % warm start  
end
```

The full function (EpsilonConstraint, with the options, the list of points and the plot of the front) is in the course companion code.

Weighted sum: `fmincon(@(x) w1*f1(x)+w2*f2(x), ...)`
without `nonl`.

- `lambda.ineqnonlin`: multiplier of $f_1 \leq \varepsilon$, here 2ε .
- Warm start: the solution for ε is a good starting point for $\varepsilon + \Delta\varepsilon$.
- `out.funcCount` counts the points where f_2 and f_1 are evaluated (including those of the solver's forward-difference gradients): it is the n_f of the sweep. In Python (SLSQP): 10 points, 60 evaluations of f_2 , 90 of f_1 (150 calls).

THE SAME IN PYTHON

```
minimize(f2, x0, bounds=b, constraints=  
{ 'type': 'ineq', 'fun': lambda x: e - f1(x) },  
method='SLSQP'); r.nfev counts the evaluations of  
f2; multipliers in r.v with trust-constr (4.2).
```

Weighted sum and ε -constraint

	weighted sum (6.3)	ε -constraint (6.4)
parameter	weights w	bounds ε
subproblem	aggregated objective	one objective + $m - 1$ constraints
<i>non-supported</i> points	no	yes*
position on the front	indirect	direct, if the constraint is active
extra information	—	multipliers (trade-off rates)
cost	1 optimization per point	1 optimization per point
many objectives	weights hard to choose	k^{m-1} grows quickly

* with subproblems solved globally (page “What can we guarantee?”).

KEY TAKEAWAYS — 6.4

1. $\min f_1$ s.t. $f_i \leq \varepsilon_i$ ($i \neq 1$): one objective is optimized; the others become **bounds**.
2. ε has a direct interpretation; with the constraint active, $f_i(x^*) = \varepsilon_i$.
3. Unlike the weighted sum, it also finds *non-supported* points (with subproblems solved globally).
4. The multipliers, $u_i = -\partial f_1^* / \partial \varepsilon_i$, are **local trade-off rates**.
5. It still costs **one optimization per point**; with m objectives, up to k^{m-1} subproblems.

So far we have obtained the front by repeating scalar problems: one point at a time.

Can we build an approximation of the front directly, in a single run?

6.5 DMS: a list of points

6.6 NSGA-II: a population

References Full list: *Course bibliography*, at the end of the course PDF.

- Haimes, Y. Y., Lasdon, L. S., Wismer, D. A., *On a bicriterion formulation of the problems of integrated system identification and system optimization*, IEEE Transactions on Systems, Man, and Cybernetics SMC-1(3):296–297, 1971.
- Chankong, V., Haimes, Y. Y., *Multiobjective Decision Making: Theory and Methodology*, North-Holland, 1983.
- Miettinen, K., *Nonlinear Multiobjective Optimization*, Kluwer Academic Publishers, 1999.

Resources

- Companion code: EpsilonConstraint. MATLAB: `fmincon(lambda.ineqnonlin, out.funcCount)`.
- Python: `minimize(f2, x0, constraints={'type':'ineq', 'fun':lambda x: e - f1(x)}, method='SLSQP');` `r.v` with `trust-constr`.